

Task 10 - Part 01: Question Answers

C# Fundamentals - Generics, Delegates, Func/Action/Predicate & Lambda Expressions

Q1. What are the benefits of using a generic sorting algorithm over a non-generic one?

A generic `SortingAlgorithm<T>` can sort **any** type - `Employee`, `int`, `string`, or anything else - using a single implementation, instead of writing a nearly identical sorting method for every type that needs sorting. This eliminates code duplication, reduces maintenance burden (a bug fix or optimization only needs to happen once), and provides compile-time **type safety**, since the compiler enforces that only elements of the specified type are used, unlike a non-generic approach that might rely on object and require unsafe casting with the risk of runtime `InvalidCastExceptions`.

Q2. How do lambda expressions improve the readability and flexibility of sorting methods?

A lambda expression lets the comparison logic for a sort be written **inline**, right where the sort is called (e.g., `(a, b) => b - a` for descending order), instead of requiring a separate, named comparer method or class defined elsewhere. This keeps the sorting intent visible at the call site, making the code easier to read at a glance, and it makes the sort highly flexible - the exact same generic `Sort` method can be reused with completely different comparison logic just by passing a different lambda, without ever touching the `Sort` method's own implementation.

Q3. Why is it important to use a dynamic comparer function when sorting objects of various data types?

Different data types (or even the same type sorted in different contexts) often need to be compared by completely different criteria - integers by value, strings by length, `Employees` by salary or by name. A **dynamic comparer function** (passed in as a delegate or lambda) lets a single generic sorting method adapt to whatever comparison logic the caller needs at that moment, rather than hardcoding one fixed comparison rule inside the sorting algorithm itself. This keeps the sorting logic decoupled from the comparison logic, so the same reusable sort works correctly no matter what "order" actually means for the data being sorted.

Q4. How does implementing `Comparable<T>` in derived classes enable custom sorting?

Implementing `Comparable<T>` (via the `CompareTo()` method) gives a class its own built-in, "natural" ordering that generic sorting utilities and built-in .NET methods (like `Array.Sort()` or `List<T>.Sort()`) can use automatically, without needing an external comparer passed in every time. When a derived class like `Manager` implements `Comparable<Manager>` and defines `CompareTo()` to compare by `Salary`, any sorting code written generically against `Comparable<T>` can sort `Manager` objects correctly without knowing anything `Manager`-specific - the comparison logic travels with the object itself.

Q5. What is the advantage of using built-in delegates like `Func<T, T, TResult>` in generic programming?

Built-in generic delegates like `Func<T, T, TResult>` remove the need to declare a custom delegate type every time you want to pass a piece of behavior around - .NET already provides a rich, standardized family of delegate types (`Func`, `Action`, `Predicate`) that cover the vast majority of common signatures. This reduces boilerplate code, makes APIs more consistent and immediately recognizable to other C# developers, and works seamlessly with lambda expressions and generic methods, since the delegate's own type parameters adapt to whatever specific types are being used at the call site.

Q6. How does the usage of anonymous functions differ from lambda expressions in terms of readability and efficiency?

An anonymous function (using the delegate keyword, e.g., `delegate(int a, int b) { return a > b; }`) and a lambda expression (e.g., `(a, b) => a > b`) are functionally very similar - both let you define behavior inline without a separate named method - but lambda expressions are far more **concise and readable**, since they drop the delegate keyword and braces for simple expressions, and they support expression-bodied syntax that maps directly onto lambda calculus notation. In terms of runtime **efficiency**, there is no meaningful difference between the two - the compiler translates both into equivalent underlying delegate instances. The practical difference is almost entirely about readability/conciseness, which is why lambda syntax has largely replaced the older anonymous method syntax in modern C# code.

Q7. Why is the use of generic methods beneficial when creating utility functions like Swap?

A generic `Swap<T>(ref T a, ref T b)` method works for **any** type - integers, strings, custom structs - using one single implementation, instead of writing `SwapInt`, `SwapString`, and so on separately for every type. This avoids repetitive code, guarantees consistent behavior everywhere swapping is needed, and automatically supports new types introduced later without requiring any additional code to be written - exactly the kind of reusable, type-safe utility generics are designed to enable.

Q8. What are the challenges and benefits of implementing multi-criteria sorting logic in generic methods?

The main **benefit** is expressive, precise control: sorting `Employee` objects first by `Salary` and then, for ties, by `Name` lets the result reflect a real-world ordering requirement that a single comparison criterion couldn't capture. The main **challenge** is that the comparer logic becomes more complex and harder to keep readable as more tie-breaking criteria are added (e.g., needing to chain several `CompareTo()` calls or nested conditionals correctly), and it's easy to introduce subtle bugs, such as inconsistent ordering, if the criteria aren't applied in a well-defined priority order. Keeping multi-criteria comparer logic in its own dedicated function (rather than an overly dense inline lambda) helps manage this complexity.

Q9. Why is the `default(T)` keyword crucial in generic programming, and how does it handle value and reference types differently?

Inside a generic method, the compiler doesn't know at compile time whether `T` will end up being a value type or a reference type, so it can't assume a literal like `0` or `null` is valid for `T`. `default(T)` solves this by returning the **appropriate default value for whatever T turns out to be** at compile/runtime: for a **value type** (like `int` or a struct), it returns the type's zero-equivalent value (`0`, `false`, or a struct with all fields at their own default); for a **reference type** (like a class or string), it returns `null`. This makes `default(T)`

essential any time generic code needs a safe, valid "empty" or "uninitialized" placeholder value, such as initializing a variable or returning a fallback result, regardless of what T ends up being.

Q10. How do constraints in generic programming ensure type safety and improve the reliability of generic methods?

A constraint (using the where clause, e.g., where T : ICloneable) restricts which types can be used as the type argument for T, guaranteeing at **compile time** that any type used with that generic class or method actually supports whatever members the generic code relies on. Without a constraint, code inside a generic method couldn't safely call ((ICloneable)item).Clone(), since a plain T offers no guarantee that Clone() exists at all. With the constraint in place, the compiler enforces the requirement at every call site, preventing an entire class of runtime errors (like a missing-method failure) that would otherwise only be discovered when the code actually runs.

Q11. What are the benefits of using delegates for string transformations in a functional programming style?

Using a delegate (e.g., a Func<string, string>) to represent "a transformation to apply to a string" lets a single generic function (like ApplyTransform(list, transformDelegate)) work with any transformation logic passed into it - uppercase conversion, reversal, trimming, or anything else - without needing to know or hardcode what that transformation actually does. This is a core functional programming idea: treating behavior itself as data that can be passed around, composed, and swapped out, which leads to highly reusable, decoupled code where the "what to do" is cleanly separated from the "how to iterate and apply it."

Q12. How does the use of delegates promote code reusability and flexibility in implementing mathematical operations?

A single method that accepts a delegate parameter representing "some operation on two integers" (e.g., Func<int, int, int>) can perform addition, subtraction, multiplication, or division - or any future operation - without needing a separate method written for each one. The core logic (taking two numbers, invoking the operation, returning the result) is written and tested **once** and reused for every operation, and new operations can be added later simply by passing a new lambda or method reference, with zero changes needed to the reusable calling code itself.

Q13. What are the advantages of using generic delegates in transforming data structures?

A generic delegate like Func<T, R> lets a single transformation function work across **any** combination of input and output types - transforming a List<int> into a List<string>, a List<Employee> into a List<decimal> (their salaries), and so on - all using one generic Transform<T, R>(List<T> source, Func<T, R> transformer) method. This avoids writing a separate transformation method for every possible pair of source/result types, keeps the transformation logic fully decoupled from the iteration logic, and remains completely type-safe, since the compiler verifies the delegate's signature matches T and R correctly at every call site.

Q14. How does Func simplify the creation and usage of delegates in C#?

Func<T, TResult> (and its variants with more input parameters) is a **built-in generic delegate type** provided by .NET, so developers no longer need to manually declare a custom delegate type (e.g., `public delegate int SquareDelegate(int x);`) before they can use it. You can immediately write `Func<int, int> square = x => x * x;` and use it right away. This removes an entire boilerplate step from working with delegates, and because Func is generic, the exact same delegate type adapts automatically to whatever input/output types are actually needed at each use site.

Q15. Why is Action preferred for operations that do not return values?

Action<T> is specifically designed to represent an operation that performs some work but returns **no value** (a void-returning operation), such as printing a string to the console. Using Action instead of Func<T, object> (or forcing a fake return value) communicates the intent clearly - both to the compiler and to other developers reading the code - that this delegate is purely about performing a side effect, not producing a result to be used afterward. It keeps the code's type signatures honest about what each delegate actually does, avoiding confusion about whether a return value exists or matters.

Q16. What role do predicates play in functional programming, and how do they enhance code clarity?

A Predicate<T> (equivalent to Func<T, bool>) represents a **condition or test** applied to a single value, returning true or false - exactly the shape needed for filtering, searching, or validating data. Using Predicate<T> as a parameter (e.g., in a `Filter(list, predicate)` method) lets the filtering *logic* be supplied by the caller while the filtering *mechanism* (looping through the list and keeping matches) stays generic and reusable. This enhances clarity because the predicate's name and signature immediately signal "this represents a true/false condition," making code that uses predicates self-documenting and easy to follow at the call site.

Q17. How do anonymous functions improve code modularity and customization?

Anonymous functions let a piece of custom behavior be defined **inline, exactly where it's needed**, and passed directly into a reusable method (like a `Filter` function) as a parameter, rather than requiring a separate, permanently named method to be declared elsewhere just to be used once. This improves modularity because the reusable method stays completely generic and unaware of the specific condition being applied, and it improves customization because callers can supply any custom logic they need on the fly - a different condition for every call - without modifying or duplicating the underlying reusable function.

Q18. When should you prefer anonymous functions over named methods in implementing mathematical operations?

Anonymous functions (or lambdas) are preferable when the logic is **short, simple, and used in only one or a few places** - defining it inline avoids the overhead of creating and naming a separate method that has no real reuse value elsewhere in the codebase, and keeps the operation visible right where it's used. A **named method** becomes the better choice when the logic is more complex, needs to be reused across many different parts of the program, benefits from a descriptive name for documentation purposes, or needs to be unit tested independently - in those cases, a named method's discoverability

and reusability outweigh the convenience of inline definition.

Q19. What makes lambda expressions an essential feature in modern C# programming?

Lambda expressions provide a concise, readable way to define behavior inline, which is foundational to writing functional-style C# code - passing logic as data to methods like LINQ's Where(), Select(), and OrderBy(), to generic sorting/filtering utilities, and to event handlers. They eliminate large amounts of verbose boilerplate that would otherwise be required (separate delegate declarations and named methods for simple, throwaway logic), and they integrate tightly with generic delegates (Func, Action, Predicate) and type inference, letting the compiler figure out parameter types automatically. This combination is central to how modern, expressive C# code (especially LINQ-heavy code) is written.

Q20. How do lambda expressions enhance the expressiveness of mathematical computations in C#?

A lambda like `(a, b) => a / b` or `x => Math.Pow(x, 2)` lets a mathematical operation be expressed directly, in a form that reads almost like standard mathematical notation, right at the point where it's used or passed as an argument. This removes the ceremony of declaring a full named method (with its own signature, braces, and return statement) just to express a simple formula, making the intent of the computation immediately clear to a reader, and it allows mathematical operations to be treated as first-class values - passed into generic calculator-style functions, stored in variables, or composed together - in a way that closely mirrors how math itself is expressed.